



Phage
SECURITY

Bounce Tech

Security Review

Conducted by:

Pyro, Lead Security Researcher

BengalCatBalu, Lead Security Researcher

Kvar, Security Researcher

YanecaB, Security Researcher

15.04.2026



Table of Contents

About Phage Security	3
Disclaimer	3
System overview	3
Executive summary	3
Overview	3
Timeline	4
Scope	4
Issues Found	5
Findings Summary	5
Findings	6
Medium Severity	6
[M-01] streamingFee updates apply retroactively to time elapsed before the change	6
[M-02] Stale lastCheckpoint when streamingFee is set to zero allows retroactive fee charging	7
Low Severity	9
[L-01] Treasury fee share can round down to zero	9
[L-02] Stale lastCheckpoint on mint() charges retroactive fees	9
[L-03] Global parameters lock funds in depreciated LeveragedTokens	10



About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to [Pyro](#).

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Bounce is a leveraged token protocol on HyperEVM where users mint and redeem ERC20 tokens representing leveraged long/short positions, with actual leverage managed off chain via Hyperliquid Perpetual Futures. Protocol fees flow to BOUNCE stakers, and an off chain automation layer bridges funds between the contracts and Hyperliquid to maintain target leverage and process queued redemptions.

Executive summary

Overview

Project Name	Bounce Tech
Repository	https://github.com/bounce-tech/bounce-contracts
Commit hash	691b3659dc817f182d95702cc9ac468ec4719aaa
Remediation	a7f0ccb5653d821893351461400582cbc30b3c65
Methods	Manual review



Timeline

Audit kick-off	08.04.2026
End of audit	09.04.2026
Remediations start	15.04.2026
Remediations end	15.04.2026

Scope

<https://github.com/bounce-tech/bounce-contracts/pull/192>
<https://github.com/bounce-tech/bounce-contracts/pull/193>
<https://github.com/bounce-tech/bounce-contracts/pull/194>
<https://github.com/bounce-tech/bounce-contracts/pull/195>
<https://github.com/bounce-tech/bounce-contracts/pull/196>
<https://github.com/bounce-tech/bounce-contracts/pull/199>
<https://github.com/bounce-tech/bounce-contracts/pull/200>
<https://github.com/bounce-tech/bounce-contracts/pull/201>

Issues Found

Severity	Count
High	0
Medium	2
Low	3

Findings Summary

ID	Title	Severity	Status
[M-01]	'streamingFee' updates apply retroactively to time elapsed before the change	Medium	Acknowledged
[M-02]	Stale 'lastCheckpoint' when 'streamingFee' is set to zero allows retroactive fee charging	Medium	Fixed
[L-01]	Treasury fee share can round down to zero	Low	Acknowledged
[L-02]	Stale 'lastCheckpoint' on 'mint()' charges retroactive fees	Low	Acknowledged
[L-03]	Global parameters lock funds in depreciated LeveragedTokens	Low	Acknowledged



Findings

Medium Severity

[M-01] streamingFee updates apply retroactively to time elapsed before the change

`LeveragedToken::_checkpoint` reads `streamingFee` from `GlobalStorage` at the moment the checkpoint executes and applies that single rate to the **entire** period since the last checkpoint:

```
// LeveragedToken.sol - _checkpoint
uint256 timeElapsed_ = to_ - $.lastCheckpoint;
uint256 percentOfYear_ = (timeElapsed_ * 1e18) / 365 days;
uint256 streamingFee_ = $.globalStorage.streamingFee(); // current rate,
no history
uint256 totalAssets_ = totalAssets();
uint256 annualFee_ = totalAssets_.mul(streamingFee_).mul($.targetLeverage);
uint256 periodFee_ = annualFee_.mul(percentOfYear_);
```

`GlobalStorage::setStreamingFee` simply overwrites the global value — it does not keep a history, does not record the change timestamp, and does not force a checkpoint on any LT:

```
// GlobalStorage.sol
function setStreamingFee(uint256 streamingFee_) external override onlyOwner {
    if (streamingFee_ == streamingFee) revert SameAsCurrent();
    if (streamingFee_ > _MAX_STREAMING_FEE) revert InvalidAmount();
    emit SetStreamingFee(streamingFee, streamingFee_);
    streamingFee = streamingFee_;
}
```

Because of this, for every LT whose `lastCheckpoint < block.timestamp` at the moment `setStreamingFee` is called, the next `_checkpoint` will charge the **new** rate for the full `timeElapsed_` — including the portion that accrued under the **old** rate.

Note that the owner cannot work around this by manually checkpointing every LT before updating the fee:

1. `LeveragedToken::checkpoint()` and `checkpoint(uint256)` are `onlyAgents` — the `GlobalStorage` owner is not authorized to call them.
2. There is no on-chain registry that enumerates all LeveragedTokens, so even an agent cannot iterate through them atomically in the same block as `setStreamingFee`.

Numerical example — the exact scenario, with `totalAssets = 1,000,000 USDC`, `targetLeverage = 3x`, and a 100-second elapsed period:

t = 0 to 50 — `streamingFee = 2%`, duration 50s

Correct fee: $1,000,000 \times 0.02 \times 3 \times (50 / 365d) \approx \9.51

Code charges: nothing yet (not checkpointed)



t = 50 — Owner calls `setStreamingFee(10%)`

t = 50 to 100 — `streamingFee = 10%`, duration 50s

Correct fee: $1,000,000 \times 0.10 \times 3 \times (50 / 365d) \approx \47.56

Code charges: nothing yet (not checkpointed)

t = 100 — `_checkpoint` fires, elapsed = 100s

Correct fee: **\$9.51 + \$47.56 = \$57.08 total**

Code charges: $1,000,000 \times 0.10 \times 3 \times (100 / 365d) \approx \95.13

The LT holders are overcharged by \$38.05 — 67% more than the correct integral. The charge is applied silently via `_payStreamingFee` at the next user interaction.

The symmetric case also occurs: lowering the rate retroactively undercharges the period before the change, quietly crediting holders fees the protocol was entitled to for the old regime.

Recommendation

Store the rate-change timestamp and the previous rate in `GlobalStorage`, and split the fee integral in `_checkpoint` whenever the last checkpoint precedes the most recent rate change:

```
// GlobalStorage.sol
uint256 public previousStreamingFee;
uint256 public streamingFeeChangedAt;

function setStreamingFee(uint256 streamingFee_) external override onlyOwner {
    if (streamingFee_ == streamingFee) revert SameAsCurrent();
    if (streamingFee_ > _MAX_STREAMING_FEE) revert InvalidAmount();
    previousStreamingFee = streamingFee;
    streamingFeeChangedAt = block.timestamp;
    emit SetStreamingFee(streamingFee, streamingFee_);
    streamingFee = streamingFee_;
}
```

In `_checkpoint`, if `lastCheckpoint < streamingFeeChangedAt <= to_`, compute the fee as two segments: `[lastCheckpoint, streamingFeeChangedAt]` at `previousStreamingFee` and `[streamingFeeChangedAt, to_]` at the current `streamingFee`. This keeps the mitigation $O(1)$ per checkpoint as long as the owner does not make more than one rate change per LT-idle period

[M-02] Stale lastCheckpoint when streamingFee is set to zero allows retroactive fee charging

In `LeveragedToken::_checkpoint`, when the calculated `periodFee_` is zero but `totalAssets_` is positive, the function returns early without updating `lastCheckpoint`:



```
// LeveragedToken.sol
uint256 annualFee_ = totalAssets_.mul(streamingFee_).mul($.targetLeverage);
uint256 periodFee_ = annualFee_.mul(percentOfYear_);
_payStreamingFee(periodFee_);
if (periodFee_ == 0 && totalAssets_ > 0) return; // @audit lastCheckpoint NOT
    updated
$.lastCheckpoint = to_;
```

This is an anti-rounding-loss mechanism: when `timeElapsed` is too small to produce a zero fee, the checkpoint stays frozen so micro-periods accumulate until a non-zero fee can be charged.

However, when the owner sets `streamingFee` to 0 via `GlobalStorage::setStreamingFee` (which is allowed — there is no `> 0` validation), `periodFee_` is **always** 0 regardless of how much time passes. The early return fires every time, and `lastCheckpoint` freezes indefinitely.

When `streamingFee` is later restored to a non-zero value, the next `_checkpoint` call computes `timeElapsed` from the stale `lastCheckpoint` and charges a retroactive streaming fee for the entire zero-fee period.

Recommendation

Update `lastCheckpoint` when `streamingFee` is zero, since there is no rounding remainder to accumulate:

```
_payStreamingFee(periodFee_);
- if (periodFee_ == 0 && totalAssets_ > 0) return;
+ if (periodFee_ == 0) {
+   if (totalAssets_ == 0 || streamingFee_ == 0) {
+     $.lastCheckpoint = to_;
+   }
+   return;
+ }
$.lastCheckpoint = to_;
```

This preserves the anti-rounding-loss behavior (when `streamingFee > 0` but the period is too short to produce 1 wei of fee, `lastCheckpoint` remains frozen), while correctly advancing the checkpoint when there is genuinely nothing to accumulate.



Low Severity

[L-01] Treasury fee share can round down to zero

In `_checkpoint()` it skips updating `lastCheckpoint` only if `periodFee_ == 0`. If `periodFee_ > 0`, it pays and updates.

But in `_payFees()` the treasury split rounds down:

```
uint256 treasuryAmount_ = feeAmount_.mul(treasuryFeeShare_);
```

So for small `feeAmount_`, `treasuryAmount_` can be 0 and all fees go to the `feeHandler`. If we are avoiding rounding to 0 in `_checkpoint()`, then rounding to 0 here should also be avoided.

Recommendation

Consider one of these:

1. Do not pay fees immediately. Accumulate them and pay once the amount is meaningful.
2. If `treasuryAmount_` rounds down to 0, skip updating the checkpoint.

[L-02] Stale lastCheckpoint on mint() charges retroactive fees

`_checkpoint()` intentionally does not update `lastCheckpoint` when `periodFee_ == 0` to avoid rounding loss.

```
if (periodFee_ == 0 && totalAssets_ > 0) return;  
$.lastCheckpoint = to_;
```

This is fine if `totalAssets_` stays flat or decreases, but not for minting where `totalAssets_` increases. If someone mints after a long period of low `totalAssets_`, the next fee accrual uses the stale timestamp and charges fees over the entire elapsed time using the **new, larger** `totalAssets`. This effectively makes the minter pay retroactive fees.

Recommendation

Consider either of these:

1. Always update `lastCheckpoint`, but calculate `periodFee_` at higher precision. Pay what you can and store the remainder to accumulate until it's large enough to pay.
2. Do not skip updating `lastCheckpoint` when a mint happens.



[L-03] Global parameters lock funds in depreciated LeveragedTokens

Two global parameters in `GlobalStorage` create independent fund-locking vectors for depreciated LeveragedTokens.

Vector 1 — `minTransactionSize` blocks entry into redemption flow.

Both `LeveragedToken::redeem` and `LeveragedToken::prepareRedeem` enforce a minimum base amount check against a single global value:

```
// LeveragedToken.sol
uint256 baseAmount_ = ltToBaseAmount(ltAmount_);
if (baseAmount_ < $.globalStorage.minTransactionSize()) revert
    BelowMinTransactionSize();
```

```
// GlobalStorage.sol
uint256 public override minTransactionSize; // single value, shared by all LTs
```

When a leveraged token's `exchangeRate` drops significantly (e.g., a 3x long LT during a major price drawdown), the USDC value of a user's LT holdings falls below `minTransactionSize`, making redemption impossible — may be even for users who hold the entire remaining supply.

Vector 2 — flat `executeRedemptionFee` blocks execution of already-prepared redemptions.

Even if a user passed the `minTransactionSize` check during `prepareRedeem`, the subsequent `_executeRedemption` recalculates `baseAmount_` at the current (potentially worse) exchange rate and subtracts a flat fee:

```
// LeveragedToken.sol — _executeRedemption
uint256 baseAmount_ = ltToBaseAmount(ltAmount_);
// ...
uint256 redemptionFee_ = _redemptionFee(baseAmount_, true);
if (baseAmount_ < redemptionFee_) return false;
```

```
// LeveragedToken.sol — _redemptionFee
uint256 percentFee_ = baseAmount_.mul(globalStorage_.redemptionFee()).mul(
    $.targetLeverage);
if (!hasFlatFee_) return percentFee_;
return percentFee_ + globalStorage_.executeRedemptionFee();
```

`executeRedemptionFee` is a fixed USDC amount set globally. If the exchange rate drops between `prepareRedeem` and `executeRedemptions`, the recalculated `baseAmount_` can become smaller than `percentFee_ + executeRedemptionFee`, causing `_executeRedemption` to silently return `false`. The user's tokens remain locked in `pendingRedemptions` — they cannot be executed, and `cancelRedeem` is only available after the timeout.



Result is that owner cannot lower `executeRedemptionFee`, `minTransactionSize` for a single struggling LT without affecting all healthy LTs.

Recommendation

Allow both `minTransactionSize` and `executeRedemptionFee` to be overridden per LeveragedToken, falling back to the global defaults when no override is set:

```
// In LeveragedTokenStorage:
uint256 minTransactionSizeOverride;    // 0 = use global
uint256 executeRedemptionFeeOverride;  // 0 = use global

// In redeem/prepareRedeem:
uint256 minSize_ = $.minTransactionSizeOverride != 0
    ? $.minTransactionSizeOverride
    : $.globalStorage.minTransactionSize();
if (baseAmount_ < minSize_) revert BelowMinTransactionSize();

// In _redemptionFee:
uint256 flatFee_ = $.executeRedemptionFeeOverride != 0
    ? $.executeRedemptionFeeOverride
    : globalStorage_.executeRedemptionFee();
return percentFee_ + flatFee_;
```

This allows the owner to lower thresholds for depreciated LTs without weakening protections on healthy ones.